

Chain of Thought (CoT) — Variations

CoT tells Copilot to *reason before it answers*. The trigger phrase is the variation. Each one changes how the reasoning unfolds.

1. Basic CoT — List then Build

The simplest form. Force enumeration before generation.

```
Before writing any code, list every edge case that  
`parseAmount(value: string): number` must handle.  
Then write the function addressing each one.
```

2. Step-by-Step CoT

The minimal trigger. Works on almost any problem.

```
Think step by step. How would you design a rate limiter  
that supports multiple algorithms (fixed window, sliding window,  
token bucket) without changing the calling code?
```

3. Plan-then-Execute

Produce a plan first. Execute only after the plan is approved.

```
First, write a numbered implementation plan for adding  
soft-delete to the User entity without breaking existing queries.  
Do not write any code yet.
```

(Review the plan, then follow up:)

```
The plan looks good. Now execute step 3 only.
```

4. Self-Critique CoT

Write it. Critique it. Improve it. Three passes in one prompt.

```
Write a function that merges two sorted arrays.  
Then critique your own implementation – identify any edge cases  
missed, any inefficiencies, or any readability issues.  
Then write an improved version based on your critique.
```

5. Failure Mode Analysis

Enumerate ways it breaks in production before writing anything.

```
Before implementing this caching layer, list every way it could  
fail or cause incorrect behaviour in production:
```

- race conditions
- cache invalidation problems
- memory leaks
- stale data scenarios
- failure when the cache is unavailable

Then write the implementation addressing each failure mode.

6. Adversarial CoT (Red Team)

Argue against your own approach first.

I want to use a single global event bus for inter-service communication in our monolith.

First, give me the strongest 3 arguments AGAINST this approach. Then, given those arguments, recommend whether to proceed and what mitigations to apply if we do.

7. Hypothesis Testing

Form a hypothesis. Test it against the code.

The bug is that orders occasionally get a 'duplicate key' error on insert. My hypothesis is that the UUID generation is not truly unique under concurrent load.

What evidence in the following code would support or refute this hypothesis? Test it systematically.

[paste code]

8. Reverse CoT (Work Backwards)

Start from the output, reason backwards to the cause.

This function is producing incorrect totals for orders that contain discount codes. Here is the wrong output:

Input: [{ price: 100, qty: 2 }, { price: 50, qty: 1 }], discount: 0.1
Expected total: 225
Actual total: 240

Work backwards from the incorrect output to identify where in the calculation the error occurs, then fix it.

9. Analogical CoT

Use an analogy to shape the reasoning.

Think of a database connection pool like a car park with a fixed number of spaces. Using that mental model, explain why the following configuration will cause timeouts under load, and what the correct settings should be:

```
max: 5
idleTimeoutMillis: 30000
connectionTimeoutMillis: 2000
```

10. Least-to-Most CoT

Solve the simplest sub-problem first, build up to the full problem.

Solve this in increasing complexity:

Step 1: Write a function that checks if a single password meets length requirements (min 8 chars).

Step 2: Extend it to also check for at least one uppercase and one digit.

Step 3: Extend it to return structured validation errors (not just true/false) so the UI can show specific messages.

Step 4: Make it configurable – accept a PasswordPolicy object so rules can change without modifying the function.

11. Contrastive CoT

Show a bad reasoning path alongside the good one. Copilot learns from the contrast.

Here are two approaches to handling the N+1 query problem.

BAD reasoning:

"Just add `.include()` everywhere and it'll be fine."

Result: loads 10,000 records when you need 10.

GOOD reasoning:

"Identify the access pattern first. Load only what the caller will use. Use a `DataLoader` for batching if called in a loop."

Apply the GOOD reasoning pattern to fix the following repository method: [paste method]

12. Socratic CoT

Copilot asks clarifying questions before answering.

I want to add full-text search to our product catalogue.
Before recommending an approach, ask me the 5 most important clarifying questions whose answers would change your recommendation.
Then, after I answer, give your recommendation.

13. Complexity-First CoT

Reason about the optimal solution before implementing.

Before writing any code:

1. What is the optimal time complexity for finding the k most frequent elements in an array?
2. What data structure achieves that complexity?
3. What are the tradeoffs vs a simpler $O(n \log n)$ approach?

Then implement the optimal solution with that reasoning as comments.

14. Test-First CoT

Reason about the tests before writing the implementation.

Before implementing `calculateShippingCost(order: Order): Money``:

1. List the test cases that a correct implementation must pass, grouped by: happy path / boundary conditions / error cases.
2. Then write the implementation that would pass all of them.

15. Security-First CoT

Reason about threats before implementing.

Before implementing the file upload endpoint:

1. List every security risk this feature introduces.
2. For each risk, state the mitigation.
3. Then write the implementation with those mitigations built in from the start.

Quick Comparison

Variation	Best for
Basic CoT	Edge case discovery
Step-by-step	General reasoning on any complex task
Plan-then-execute	Large changes where you want to review the approach first
Self-critique	Getting a better answer than the first pass

Failure mode analysis	Anything that touches production data or state
Adversarial	Architecture decisions — stress-test your own idea
Hypothesis testing	Debugging — structured root cause analysis
Reverse CoT	Bug fixing from observed wrong output
Analogical	Explaining or reasoning about complex abstractions
Least-to-most	Teaching, or building complex logic incrementally
Contrastive	Correcting a pattern Copilot keeps repeating badly
Socratic	When you're not sure what information Copilot needs
Complexity-first	Algorithm design, performance-sensitive code
Test-first	TDD workflow, acceptance criteria from requirements
Security-first	Auth, file handling, external input, payment flows

Combining Variations

Variations stack. The most powerful patterns combine two:

```
/* Plan-then-Execute + Failure Mode Analysis */
```

```
Plan the implementation of our webhook signature verification.  
Before the plan, list every way signature verification can be  
bypassed or fail silently. Address each in the plan.  
Then execute step 1 of the plan only.
```

```
/* Self-Critique + Security-First */
```

```
Write the password reset flow.  
Then critique it as a penetration tester looking for  
account takeover vulnerabilities.  
Then produce a final version with all vulnerabilities addressed.
```